Volume 05 | Scheduling, Station Mapping, and Travel Graph

Deep explanation of service recommendations and graph node identity.

1. Scheduling input/output

`optimize_station_schedule` takes station, primary/all lines, hour, weekday, total estimated footfall, workday and service-open. It selects line shares from JSON or equal-share fallback, splits demand, then invokes `optimize_line_schedule` for each corridor. The output carries baseline/recommended trains per hour and headway, clearance capacity, utilization, wait estimate, fleet constraints, risk badge and narrative.

2. Frequency and capacity equations

Frequency = `60 / headway_minutes`. Fleet-supported maximum frequency = `floor(fleet_size * 60 / turnaround_minutes)`. Signaling-supported maximum = `floor(60 / min_signal_headway)`. The absolute maximum is the smaller. Policy maximum headway sets a minimum frequency. Peak on weekday uses a 65% assumed through-load, leaving 35% of comfort capacity for boarding; off-peak uses 40%, leaving 60%. Clearance budget per train is `max(int(comfort_capacity * (1-through_load)),250)`. Existing boarding clearance per hour is trains/hour times that budget.

If demand exceeds existing clearance, target frequency is demand divided by clearance budget. Some low-demand off-peak periods can reduce frequency; otherwise keep baseline. Clamp to fleet/signaling/policy constraints, then round to closest configured practical headway. Utilization = demand / final recommended clearance capacity x 100. Badge thresholds then classify low/moderate/high/critical. Values are configurable assumptions and should not be represented as verified operating standards.

3. What utilization is not

It estimates demand against modeled clearance available for new boarding. It is not current occupancy, load on an individual train, passenger count already aboard, platform density, or observed boarding/alighting. The through-load fraction approximates seats/space occupied before reaching the station but is a fixed scenario assumption, not a train telemetry feed.

4. Station-name mapping pipeline

Canonical station table -> topology labels -> normalization -> aliases -> canonical labels -> graph nodes.

- `_load_stations` reads station string and coordinates; normalization removes footnote symbols, lowercases, smooths dash variants, removes punctuation and collapses one duplicate-space case.
- `_match_topology_to_canonical` collects labels from all ordered route segments and interchange keys. Exact normalized match wins; otherwise `ALIASES` maps selected normalized label to canonical spelling.
- Current audit: 243 topology labels match 243 cleaned records; no unmatched labels in this snapshot. This should become an automated invariant; future edits can regress it.
- Normalization dictionary can silently overwrite collisions; stable station IDs and collision audit are safer than labels as primary keys.

5. Why graph node is `(station, line)`

One physical interchange is multiple possible transit states. Example: `(Rajiv Chowk, Yellow Line)` and `(Rajiv Chowk, Blue Line)`. If one station-only node were used, switching lines at the interchange could happen at zero cost. If line nodes existed without transfer edges, the network would be disconnected. The `INTERCHANGES` map creates explicit bidirectional edges between line states with a 5-minute network-wide penalty. Multi-source Dijkstra starts all origin line states at zero, so it does not charge an initial transfer simply to begin at the selected origin.

6. Route edges and weights

For each ordered route segment, consecutive stations become bidirectional edges. Weight: if both coordinates are usable, haversine km / 32 km/h x 60 with at least 1 minute; if computed duration exceeds 8 minutes, treat coordinates as implausible and use 2.5-minute fallback. Missing coordinates or a coordinate pair shared by more than two stations also triggers fallback. Haversine is straight-line distance; it approximates adjacent route distance and ignores track geometry, dwell differences and timetable. Branches are separate route sequences; interchanges connect only where explicitly listed.

7. Dijkstra details

Initialize distance zero for source line nodes; push to min-heap. Pop smallest tentative cost; skip visited states; relax neighbors if `d+w` improves. Since all weights are nonnegative, settled distances are final. After search, collapse line states by physical station with minimum distance and remove origin. Complexity `O((V+E)log V)`. Graph is small and search is run on requests. Output is estimated travel time, not necessarily a line itinerary; route API currently attaches a primary line label to each destination.

Exercises

Sketch Rajiv Chowk line states and its transfer edge. Show a route crossing one interchange and sum edge weights. Explain how an unmatched alias can remove reachability. Recalculate clearance if through-load fraction changes. Explain why route duration estimates cannot be marketed as timetable accuracy.